



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ASSIGNMENT REPORT

SMART CAMPUS BIKE SHARING MANAGEMENT SYSTEM
USING C++ OBJECT-ORIENTED PROGRAMMING

Course Code: DSA0109

Course Name: Object Oriented Programming with C++

Reshma K-192421256

Dhanushree S-192421218

Sunitha M (192424157)

Table of Contents

S. No.	Titles	Page No.
1	Introduction	1
2	Problem Statement	2
3	Objectives	3
4	Requirements and Environment Used	4
5	Design / Proposed Solution	5
6	System Architecture	6
7	Class Design / UML Diagram	7
8	Object-Oriented Programming Concepts Used	8
9	Algorithm / Pseudocode	9
10	Main System Flowchart	10
11	Student Registration Flowchart	11
12	Bike Registration Flowchart	13
13	Bike Rental Flowchart	14
14	Bike Return and Maintenance Flowchart	15
15	Constructors and Object Initialization	17
16	Operator Overloading	19
17	Runtime Polymorphism	20
18	Implementation / Source Code	21
19	Program Menu	22
20	Test Cases	23
21	Execution Screenshots / Results	25
22	Campus Bike-Sharing Activity Report	27
23	Analysis and Discussion	30
24	Advantages	31
25	Limitations and Future Enhancements	33
26	Individual Contribution	36
27	Conclusion	38
28	References	41
29	GitHub Repository Details	43

1. Introduction

The Smart Campus Bike Sharing Management System is a menu-driven C++ application designed to manage bicycles that are shared among students inside a university campus. The system treats the bicycle fleet as a managed resource rather than as a simple list of vehicles. It keeps track of students, bicycles, rental transactions and operational states such as availability, active rental and maintenance.

A campus commonly contains academic blocks, hostels, libraries, laboratories, administrative buildings, sports facilities and cafeterias. Students and staff may need to move between these locations several times a day. A shared bicycle service can reduce unnecessary walking time and provide an efficient short-distance transportation option. Once the fleet becomes larger, however, manually maintaining bicycle IDs, student details and rental records becomes increasingly difficult.

The proposed application solves this management problem through Object-Oriented Programming. The real-world entities are mapped to C++ classes. A Student object stores the identity of a campus user, a Bike object represents common bicycle information and a Rental object records a borrowing transaction. The Bike class is designed as an abstract base class, while StandardBike, ElectricBike and MountainBike represent specialized bicycle categories.

The project is also designed to demonstrate the academic requirements of CO4. Default, parameterized and copy constructors show different forms of object initialization. A virtual destructor demonstrates safe cleanup when objects are managed through base-class pointers. Operator overloading gives user-defined Bike objects meaningful operations such as rate comparison, charge calculation and formatted output.

The system therefore combines a practical campus-management scenario with important C++ concepts. It demonstrates how classes can be used to encapsulate data, how inheritance reduces repeated code, how virtual functions provide dynamic behaviour and how a menu-driven application can coordinate multiple related objects.

1.1 Need for the System

The need for the system arises from the gap between a simple bicycle-sharing idea and the administrative effort required to operate it consistently. Every rental changes the state of a bicycle, creates a transaction and establishes a relationship between a student and the bicycle. Similarly, a maintenance event must make a bicycle unavailable and a service event must make it available again. Automating these state transitions reduces manual errors.

1.2 Academic Relevance

From an academic perspective, the project provides a concrete environment in which OOP concepts can be explained rather than demonstrated in isolation. The same Bike* pointer can refer to several derived bike categories, constructors establish initial state, overloaded operators provide object-level behaviour, and the BikeSystem class coordinates the overall application.

2. Problem Statement

In a manually managed campus bicycle-sharing service, administrators may use notebooks, spreadsheets or separate informal records to track which bicycle is available, who has borrowed it, how long it has been used and whether it requires maintenance. As the number of users and bicycles increases, this approach can produce duplicate records, incorrect availability information, forgotten active rentals and difficulty in generating usage statistics.

The required solution is a C++ application that automates the core lifecycle of a shared bicycle. A student must be registered before using the service. A bicycle must have a unique ID and a valid category. Only an AVAILABLE bicycle can be rented. A rental should record the student, bicycle, number of hours and calculated charge. Returning a bicycle should close its active rental and make the bicycle available again.

The application must also distinguish specialized bicycle behaviour. Standard, electric and mountain bicycles may share common state but can have different descriptions and charge calculations. This difference provides a natural reason for inheritance and runtime polymorphism.

Finally, the implementation must explicitly demonstrate constructors, a destructor and operator overloading. The resulting system should therefore be both functionally meaningful and aligned with the OOP learning outcome.

3. Objectives

- Design and implement a complete menu-driven campus bicycle-sharing prototype in C++.
- Represent students, bicycles and rental transactions using classes and objects.
- Create an abstract Bike base class containing common bicycle state and behaviour.
- Implement StandardBike, ElectricBike and MountainBike using inheritance.
- Demonstrate encapsulation by keeping important data members protected or private and changing state through member functions.
- Demonstrate abstraction using a pure virtual getDescription() function.

- Demonstrate runtime polymorphism through Bike* pointers and virtual functions.
- Implement default, parameterized and copy constructors.
- Implement a virtual destructor and explicit cleanup of dynamically allocated bike objects.
- Overload +, > and << for operations that are meaningful in the bike-sharing domain.
- Provide student registration, bicycle registration, listing, rental, return, maintenance and service operations.
- Maintain rental history for transactions created during program execution.
- Generate a final activity report showing fleet state and rental statistics.
- Validate duplicate IDs, invalid categories, invalid rates, invalid hours and unavailable bicycles.
- Prepare a test plan containing valid, invalid and boundary-style scenarios.
- Organize the source code, diagrams, screenshots and report for GitHub submission.

4. Requirements and Environment Used

Component	Specification / Use
Programming Language	C++11 or later
Compiler	GCC / MinGW / Clang / MSVC
IDE	VS Code / Code::Blocks / Visual Studio / equivalent
Operating System	Windows / Linux / macOS
Version Control	Git and GitHub
Libraries	iostream, iomanip, string, vector
Input	Keyboard / console
Output	Console
Data Structures	vector<Student>, vector<Rental>, vector<Bike*>
Memory	Dynamic allocation for derived Bike objects

4.1 Hardware Requirements

The prototype has modest hardware requirements because it is a console application and does not perform heavy computation. A standard student laptop or desktop with a modern processor, sufficient RAM for a C++ compiler and a few hundred megabytes of free storage is adequate.

4.2 Software Requirements

The source requires a C++ compiler supporting C++11 or later. The program uses standard library facilities, so no external framework is required. Git is recommended for version control and GitHub is recommended for source-code submission.

4.3 Compilation Command

```
g++ -std=c++11 main.cpp -o bike_system  
./bike_system
```

On Windows using MinGW, the executable can be launched as bike_system.exe. In an IDE, the equivalent Build and Run command can be used.

5. Design / Proposed Solution

The proposed system is divided into logical responsibilities so that each part of the program has a clear purpose. The design follows a domain-oriented approach: objects represent the important campus entities, while BikeSystem coordinates operations involving several objects.

5.1 Student Management

Student management stores the identity of people who are permitted to use the bicycle-sharing service. The system checks the student ID before adding a new record. A duplicate ID is rejected because the ID is treated as the unique identifier for the student.

5.2 Bike Management

Bike management maintains the fleet. Each bicycle has a unique ID, model name, status and hourly rate. The bike type determines which derived class is created. The common state remains in Bike while specialized behaviour is implemented by the derived classes.

5.3 Rental Management

Rental management validates the student and bicycle, checks that the requested number of hours is positive and confirms that the bicycle is AVAILABLE. A successful rental changes the bike status to RENTED and creates a Rental object.

5.4 Return Management

Return management searches for the requested bicycle and an active Rental record. When found, the bike returns to AVAILABLE status and the transaction is closed. If no active rental exists, the system reports the error without changing the state.

5.5 Maintenance Management

A bicycle can be placed in MAINTENANCE status when it requires inspection or repair. A bicycle under maintenance is not available for rental. After service, the status is changed back to AVAILABLE.

5.6 Reporting

The report module traverses the stored objects and calculates total bicycles, available bicycles, rented bicycles, maintenance bicycles, registered students, rental records and accumulated rental charges.

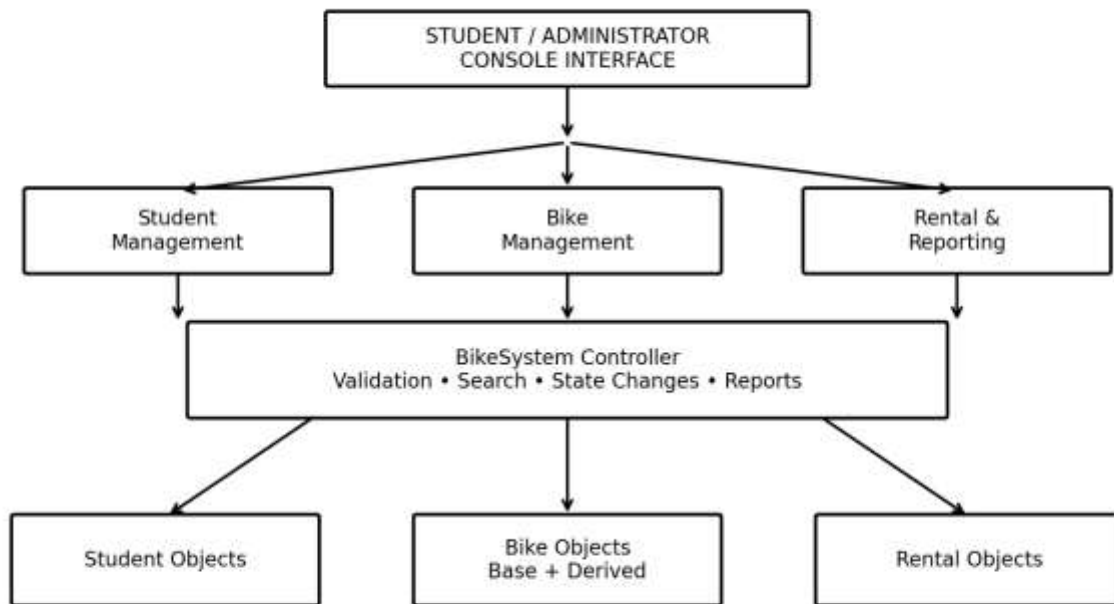


Figure 1. Proposed system architecture

6. System Architecture

The architecture contains three practical layers. The console interface receives user commands and displays results. The BikeSystem controller contains the application-level operations and coordinates collections of objects. The domain model contains Student, Bike and Rental objects.

6.1 Presentation Layer

The presentation layer is the menu displayed in the console. It is intentionally simple so that the focus remains on object-oriented design. Each menu choice maps to a public BikeSystem function.

6.2 Controller / Management Layer

BikeSystem is the central management class. It stores vectors of students, rentals and Bike pointers. It performs searches, validation, object creation, state changes and report generation.

6.3 Domain Layer

The domain layer contains the classes that model the problem. Bike provides the common interface; derived classes provide specialized behaviour. Student identifies the user and Rental stores the relationship between a student and a bicycle for a particular transaction.

6.4 State Model

```
AVAILABLE --rent()--> RENTED
RENTED    --returnBike()--> AVAILABLE
AVAILABLE --setMaintenance()--> MAINTENANCE
MAINTENANCE --serviceBike()--> AVAILABLE
```

This state model is important because the system must never treat a RENTED or MAINTENANCE bicycle as freely available.

7. Class Design / UML Diagram

The class hierarchy places shared bicycle information in the abstract Bike class. StandardBike, ElectricBike and MountainBike inherit this common structure. The Student and Rental classes are independent domain entities, while BikeSystem aggregates collections of these objects.

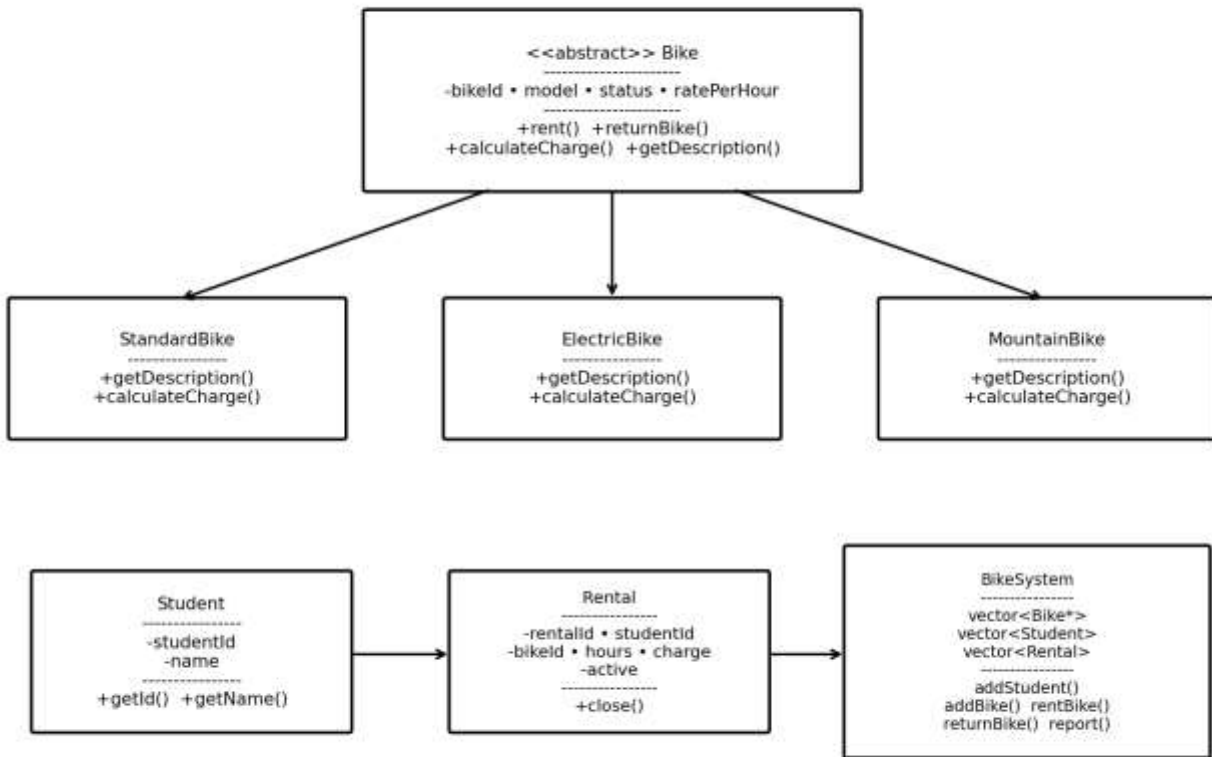


Figure 2. UML-style class hierarchy and associations

7.1 Bike Base Class

Bike contains protected data members for bikeId, model, status and ratePerHour. It defines common operations such as rent(), returnBike(), setMaintenance(), serviceBike() and calculateCharge(). The pure virtual getDescription() function makes the class abstract.

7.2 Derived Bike Classes

StandardBike represents a regular campus bicycle. ElectricBike represents a bicycle with electric assistance and can apply an additional service/battery fee in its overridden charge calculation. MountainBike represents a trail-oriented bicycle and can apply a different service fee.

7.3 Student and Rental

Student stores studentId and name. Rental stores rentalId, studentId, bikeId, hours, charge and active status. These classes keep user and transaction information separate from the bicycle's physical state.

7.4 BikeSystem

BikeSystem owns the collections used by the program and is responsible for high-level workflows. It also owns the dynamically allocated Bike objects and therefore releases them in its destructor.

8. Object-Oriented Programming Concepts Used

8.1 Encapsulation

Encapsulation means grouping data and the functions that operate on that data into classes while controlling direct access. Bike keeps its state inside the class and exposes controlled operations such as `rent()`, `returnBike()`, `setMaintenance()` and `serviceBike()`. Rental also hides its internal active state and exposes `close()`. This prevents random external code from changing a bike status without following the intended workflow.

8.2 Inheritance

Inheritance allows the derived bike categories to reuse the common properties and operations of Bike. Instead of creating three unrelated classes with repeated ID, model, status and rate members, the common implementation is written once in Bike.

8.3 Abstraction

Bike is abstract because `getDescription()` is declared as a pure virtual function. The base class defines what a concrete bike must provide without forcing the base class itself to represent a specific bicycle category.

8.4 Runtime Polymorphism

BikeSystem stores `Bike*` pointers. A pointer can refer to `StandardBike`, `ElectricBike` or `MountainBike`. When a virtual function is invoked, C++ performs dynamic dispatch and selects the implementation belonging to the actual object.

8.5 Constructors

Constructors initialize objects at creation time. The project demonstrates default, parameterized and copy constructors, giving a direct illustration of different initialization mechanisms.

8.6 Destructor

Bike has a virtual destructor and BikeSystem deletes each dynamically allocated Bike object. The virtual destructor is important when a derived object is deleted through a Bike pointer.

8.7 Pointers

Pointers are used to store dynamically allocated derived Bike objects in a common vector<Bike*> collection. This is the mechanism that makes the polymorphic design possible.

8.8 this Pointer

The this pointer refers to the current object. It is useful when a member function needs to explicitly identify the object whose state is being accessed or changed.

8.9 Operator Overloading

The project overloads +, > and <<. These operators are connected to bike behaviour: charge calculation, hourly-rate comparison and formatted output.

8.10 Composition / Association

BikeSystem maintains collections of Student, Rental and Bike objects and coordinates their interactions. A Rental links a student ID with a bike ID, creating a logical association between the entities.

9. Algorithm / Pseudocode

The main algorithm is designed around a repeated menu loop. Every operation first validates its input and then changes the relevant object state only if the operation is valid.

START

 Initialize BikeSystem

 Initialize empty Student, Bike and Rental collections

REPEAT

 Display menu

 Read choice

 IF Register Student

 Read ID and name

 Search for duplicate ID

 IF duplicate

 Reject

 ELSE

Create Student and store it
END IF

ELSE IF Add Bike

Read bike ID, model, type and hourly rate
Validate unique ID, type and positive rate
Create selected derived Bike object
Store Bike pointer

ELSE IF List Bikes

Traverse all Bike pointers
Display each bike using operator <<

ELSE IF Rent Bike

Read student ID, bike ID and hours
Find both objects
Validate IDs, hours and AVAILABLE status
Call virtual rent()
Calculate charge
Create Rental record

ELSE IF Return Bike

Read bike ID
Find bike and active rental
Call returnBike()
Close rental

ELSE IF Maintenance

Find bike
Set MAINTENANCE status

ELSE IF Service

Find bike

Set AVAILABLE status

ELSE IF History

Display all Rental records

ELSE IF Operator Demo

Demonstrate +, > and <<

ELSE IF Copy Demo

Create an object and copy it

ELSE IF Report

Traverse bikes and rentals

Calculate statistics

Display report

UNTIL choice = Exit

Delete dynamically allocated Bike objects

STOP

10. Main System Flowchart

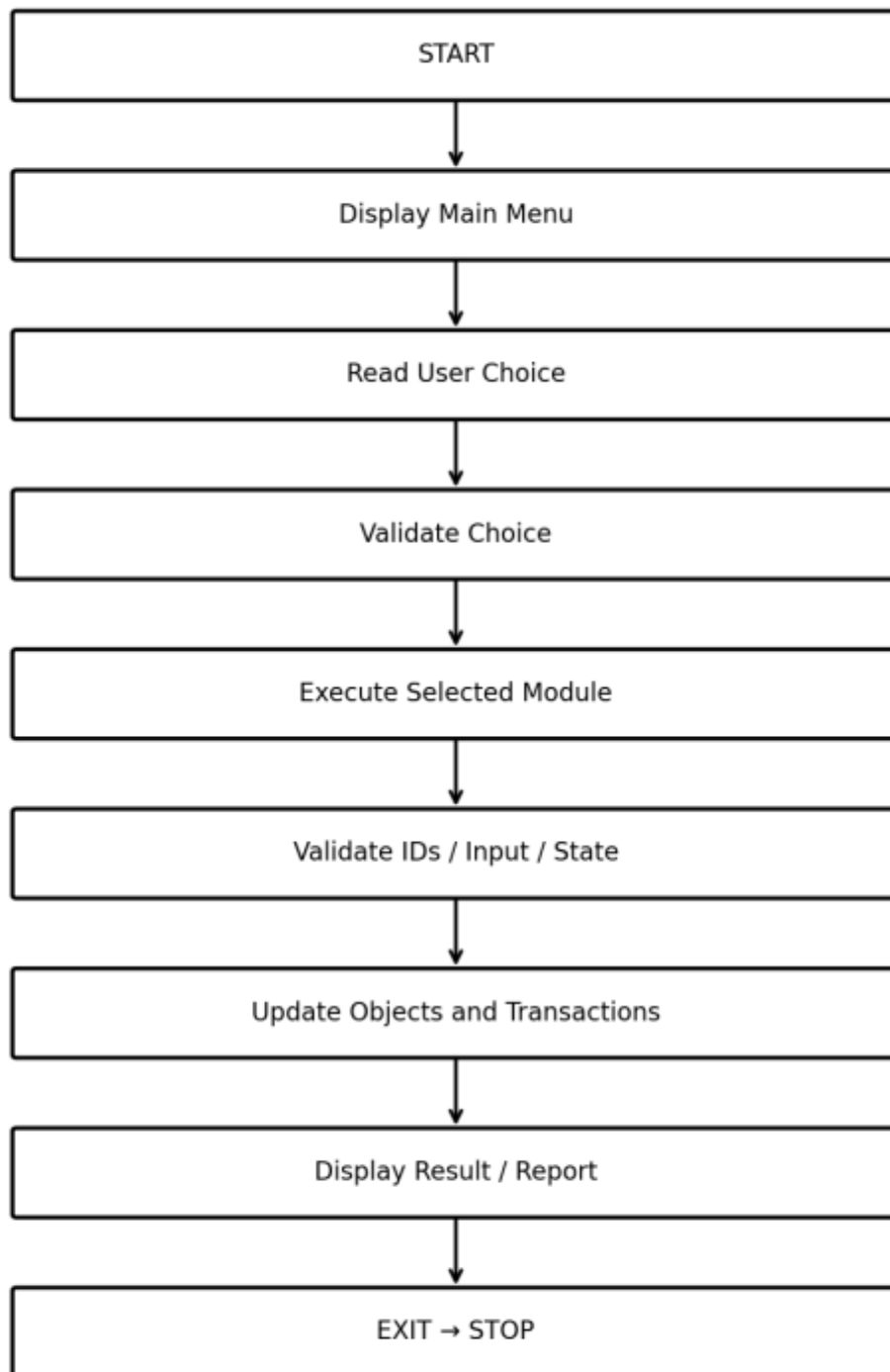


Figure 3. Main system flowchart

The main flow begins with program initialization and repeatedly returns to the menu after completing an operation. This structure ensures that the user can perform multiple registrations, rentals, returns and reports during a single execution.

11. Student Registration Flowchart

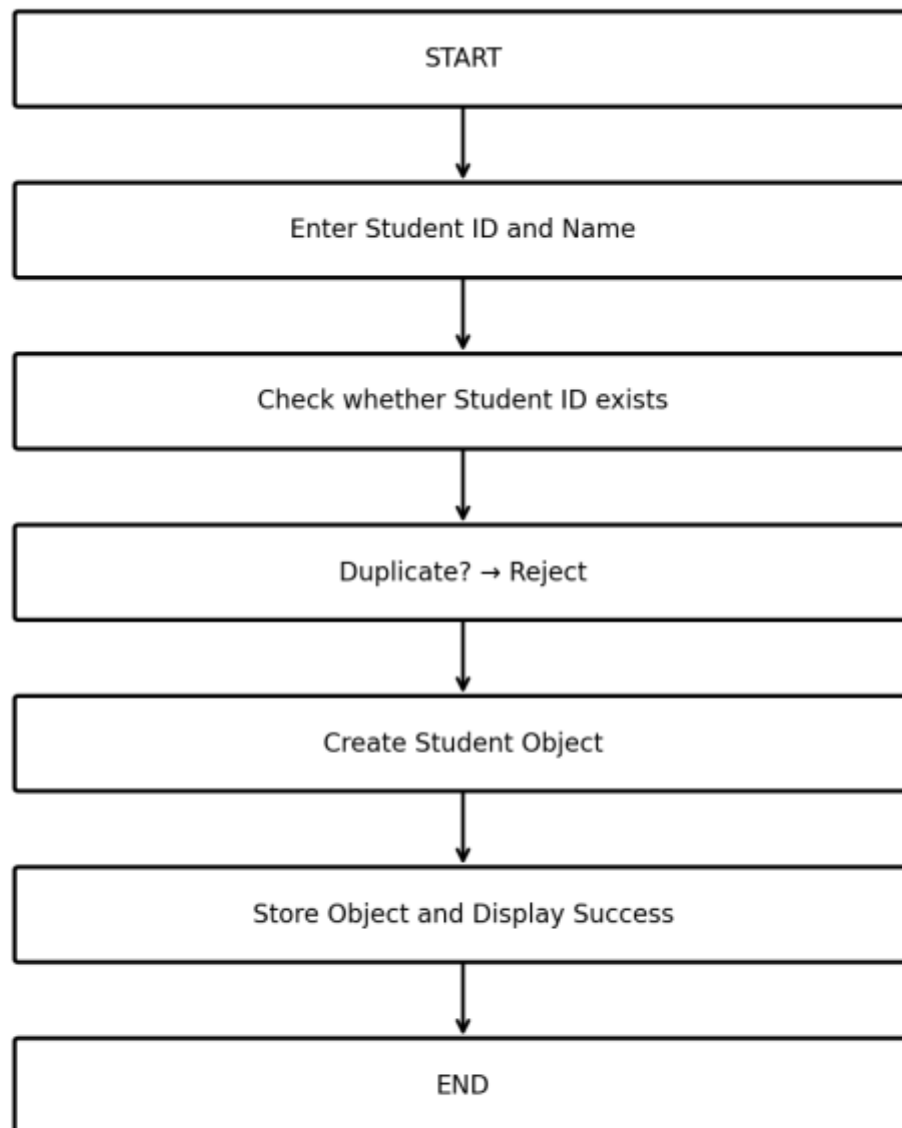


Figure 4. Student registration flowchart

The registration workflow treats studentId as a unique key. A duplicate check is performed before the Student object is inserted into the vector. This prevents ambiguous student records.

12. Bike Registration Flowchart

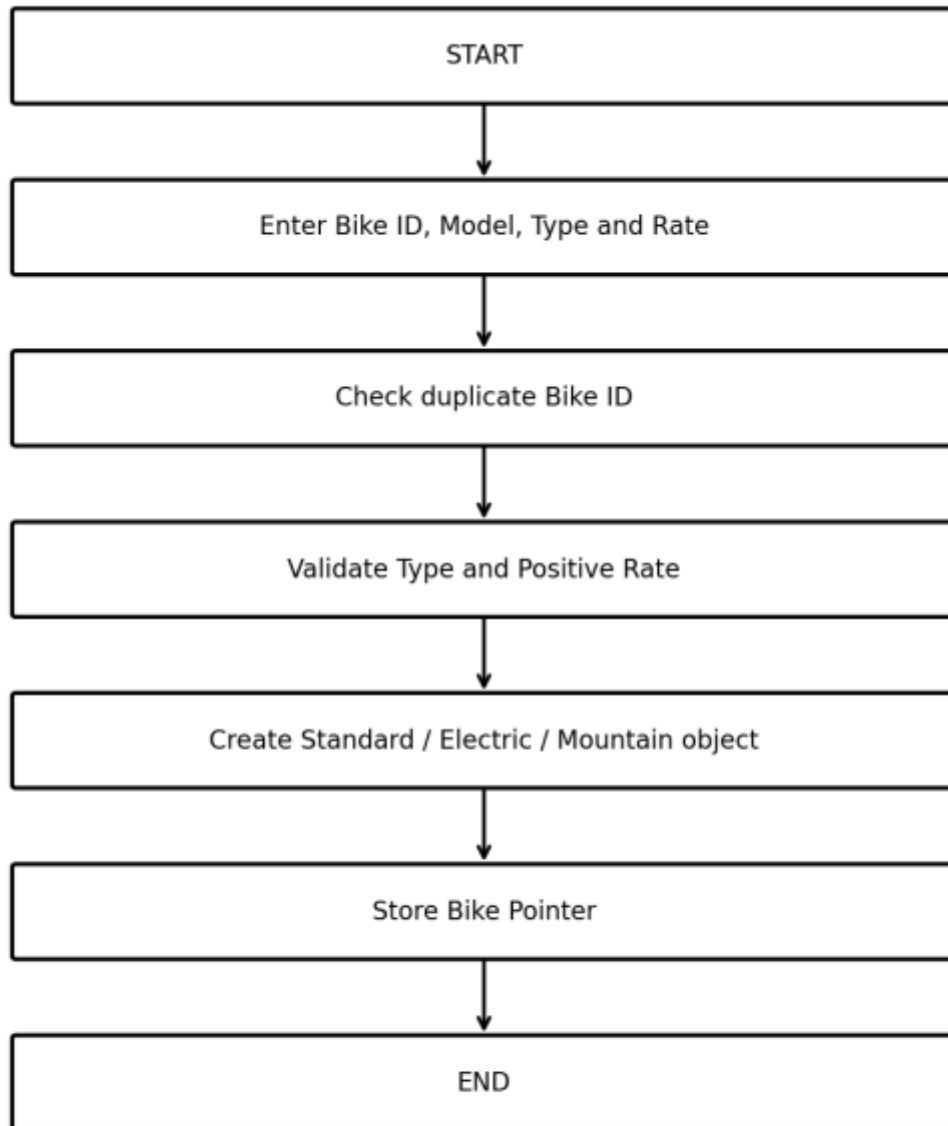


Figure 5. Bike registration flowchart

The bike-registration workflow validates both the identifier and the category. The selected type controls which derived class is dynamically allocated. A non-positive hourly rate is rejected because it would make rental charging invalid.

13. Bike Rental Flowchart

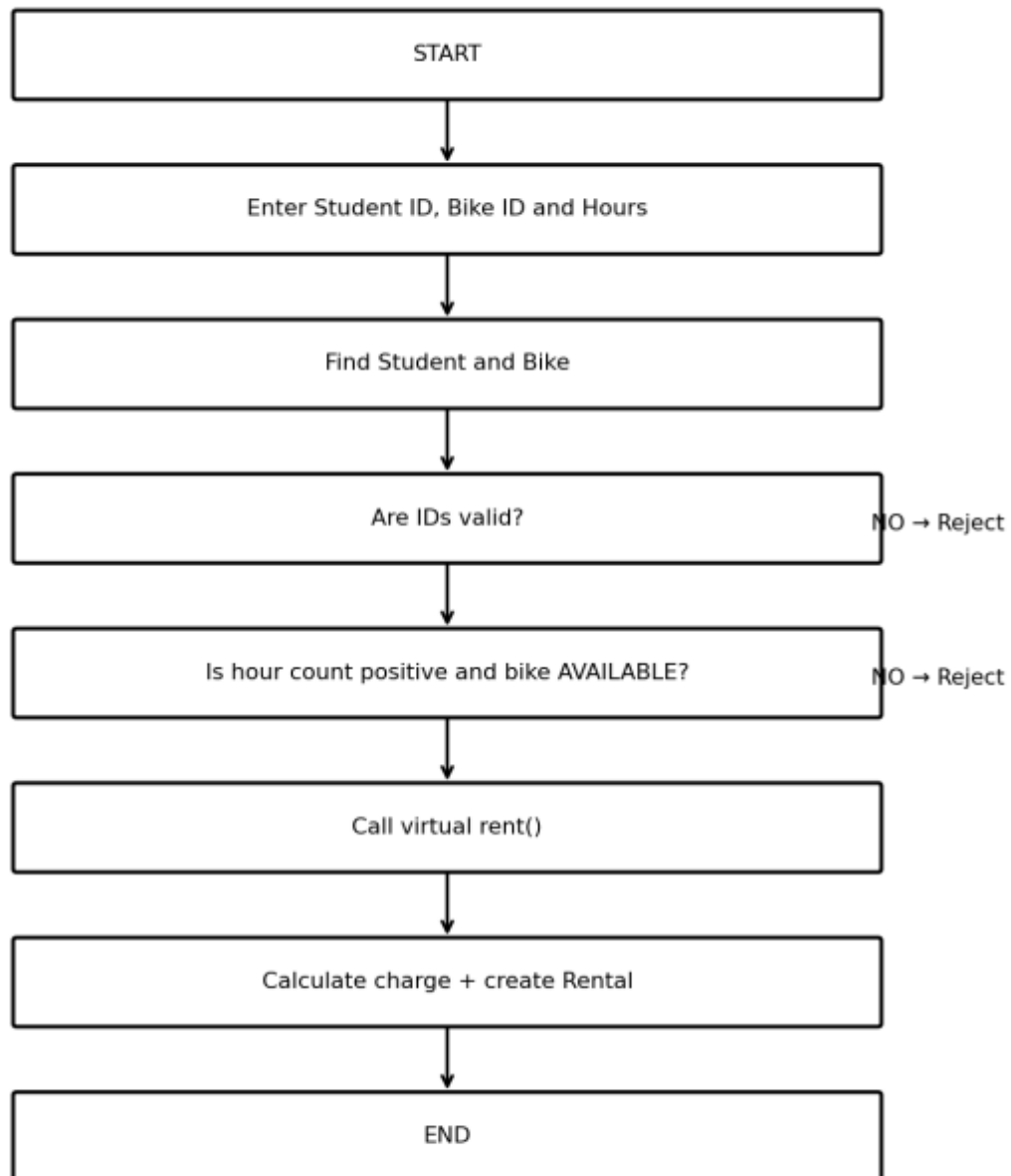


Figure 6. Bike rental flowchart

Rental is the most important state-changing operation. It verifies both the user and the resource before calling the virtual `rent()` function. Only after a successful state change is a Rental record created.

14. Bike Return and Maintenance Flowchart

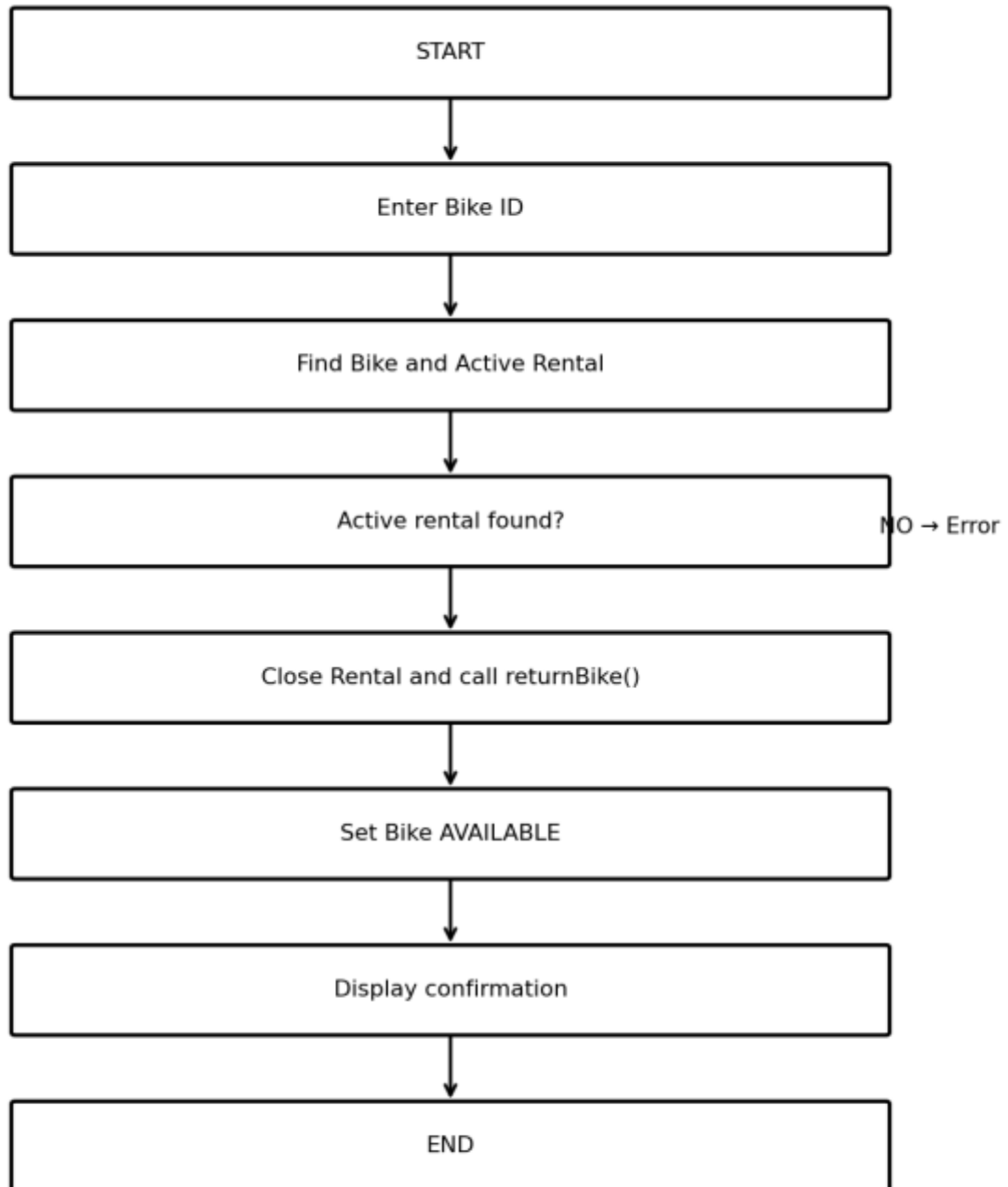


Figure 7. Bike return flowchart

The return operation searches for an active transaction. A maintenance operation is conceptually similar but changes the bike into MAINTENANCE status instead of closing a rental. Service returns the bicycle to AVAILABLE.

15. Constructors and Object Initialization

Constructors are special member functions used to establish the initial state of objects. In this project, constructor usage is especially important because the system creates different kinds of objects dynamically and through vectors.

15.1 Default Constructor

The default constructor creates a valid object using safe placeholder values. It is useful when an object must exist before actual values are assigned.

```
Bike()  
    : bikeId("NA"), model("Unknown"),  
      status("AVAILABLE"), ratePerHour(0.0) {}
```

15.2 Parameterized Constructor

The parameterized constructor accepts the values supplied during registration and immediately creates a correctly initialized object.

```
Bike(string id, string m, double rate)  
    : bikeId(id), model(m),  
      status("AVAILABLE"), ratePerHour(rate) {}
```

15.3 Copy Constructor

The copy constructor initializes one object from another. The implementation creates a distinguishable copy identifier so the demonstration can show both objects without presenting them as the same registered bicycle.

```
Bike(const Bike& other)  
    : bikeId(other.bikeId + "_COPY"),  
      model(other.model),  
      status(other.status),  
      ratePerHour(other.ratePerHour) {}
```

15.4 Destructor

Because Bike objects are created dynamically, the system must release them. Bike declares a virtual destructor, and BikeSystem deletes every pointer in its destructor.

```
virtual ~Bike() {}
```

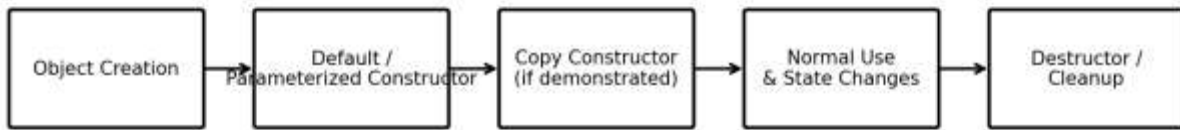


Figure 8. Object lifecycle showing initialization, use and cleanup

The lifecycle diagram shows that constructors establish initial state, member functions modify state during normal use and the destructor participates in cleanup when the object lifetime ends.

16. Operator Overloading

Operator overloading allows standard C++ operators to be defined for user-defined classes. The important design principle is that the overloaded operator should have a clear relationship to the class rather than being added only as a syntax exercise.

Operator	Meaning in Bike System	Example
+	Calculate the rental charge for a supplied number of hours.	bike + 2
>	Compare the hourly rates of two bicycles.	bike1 > bike2
<<	Print a formatted bicycle record.	cout << bike

```
double operator+(int hours) const {  
    return calculateCharge(hours);  
}
```

```

}

bool operator>(const Bike& other) const {
    return ratePerHour > other.ratePerHour;
}

friend ostream& operator<<(ostream& out, const Bike& b) {
    out << left
        << setw(10) << b.bikeId
        << setw(18) << b.model
        << setw(15) << b.getDescription()
        << setw(15) << b.status
        << fixed << setprecision(2)
        << b.ratePerHour;
    return out;
}

```

The + operator delegates to the virtual calculateCharge() function, which is significant because the overloaded operator can still respect derived-class behaviour. The > operator compares rates and the << operator makes list and report output concise and readable.

17. Runtime Polymorphism

Runtime polymorphism enables a common interface to operate on different derived objects. In this project, Bike is the common interface and Bike* is the pointer type used by BikeSystem.

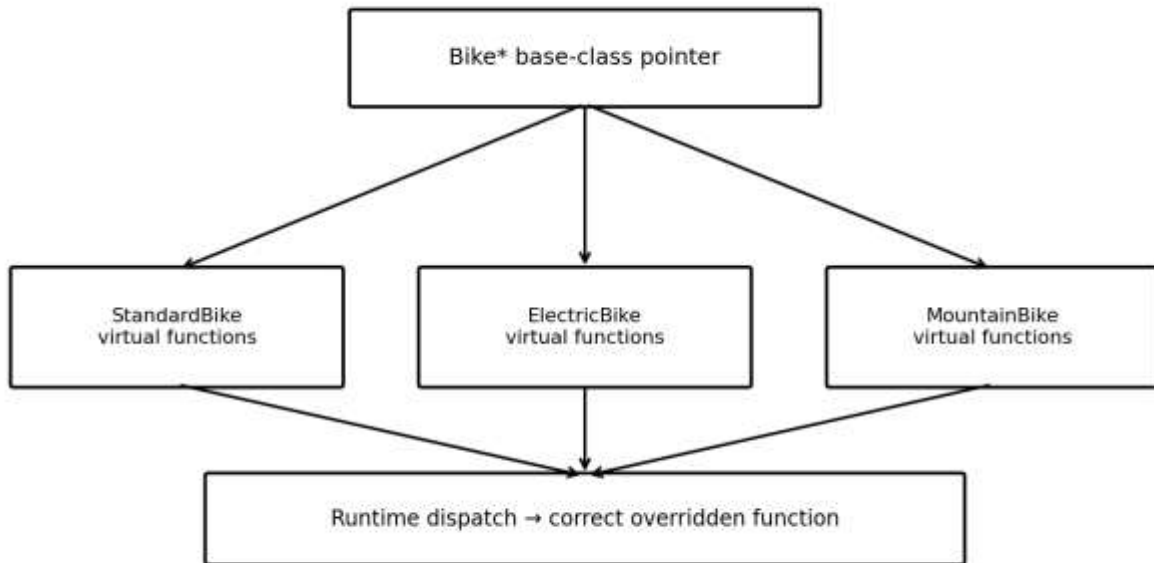


Figure 9. Runtime polymorphism through Bike pointers

Base Pointer	Actual Object	Virtual Behaviour
Bike*	StandardBike	Standard description and base charge
Bike*	ElectricBike	Electric description and overridden charge
Bike*	MountainBike	Mountain description and overridden charge

For example, when BikeSystem executes `b->calculateCharge(hours)`, the function that belongs to the actual derived object is selected at runtime. This means the rental module does not need separate top-level logic for every bike category.

```

Bike* b1 = new StandardBike("B101", "Hero", 20);
Bike* b2 = new ElectricBike("B102", "Volt", 40);
Bike* b3 = new MountainBike("B103", "Rockrider", 30);

```

```

cout << b1->getDescription();
cout << b2->getDescription();
cout << b3->getDescription();

```

18. Implementation / Source Code

The implementation below is organized around the domain classes and the controller. The complete source file should also be stored in GitHub as main.cpp.

18.1 Header Files

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;
```

18.2 Bike Base Class

```
class Bike {
protected:
    string bikeId, model, status;
    double ratePerHour;

public:
    Bike()
        : bikeId("NA"), model("Unknown"),
          status("AVAILABLE"), ratePerHour(0.0) {}

    Bike(string id, string m, double rate)
        : bikeId(id), model(m),
          status("AVAILABLE"), ratePerHour(rate) {}

    Bike(const Bike& other)
        : bikeId(other.bikeId + "_COPY"),
          model(other.model),
          status(other.status),
          ratePerHour(other.ratePerHour) {}

    virtual ~Bike() {}
```

```

string getId() const { return bikeId; }
string getStatus() const { return status; }
double getRate() const { return ratePerHour; }

virtual string getDescription() const = 0;

virtual double calculateCharge(int hours) const {
    return ratePerHour * hours;
}

virtual bool rent() {
    if (status != "AVAILABLE") return false;
    status = "RENTED";
    return true;
}

virtual bool returnBike() {
    if (status != "RENTED") return false;
    status = "AVAILABLE";
    return true;
}

void setMaintenance() { status = "MAINTENANCE"; }
void serviceBike() { status = "AVAILABLE"; }

double operator+(int hours) const {
    return calculateCharge(hours);
}

bool operator>(const Bike& other) const {
    return ratePerHour > other.ratePerHour;
}

```

```

friend ostream& operator<<(ostream& out, const Bike& b) {
    out << left
        << setw(10) << b.bikeId
        << setw(18) << b.model
        << setw(15) << b.getDescription()
        << setw(15) << b.status
        << fixed << setprecision(2)
        << b.ratePerHour;
    return out;
}
};

```

18.3 Derived Classes

```

class StandardBike : public Bike {
public:
    StandardBike() : Bike() {}
    StandardBike(string id, string m, double rate)
        : Bike(id, m, rate) {}
    StandardBike(const StandardBike& other)
        : Bike(other) {}

    string getDescription() const override {
        return "Standard";
    }
};

```

```

class ElectricBike : public Bike {
public:
    ElectricBike() : Bike() {}
    ElectricBike(string id, string m, double rate)
        : Bike(id, m, rate) {}

```

```

string getDescription() const override {
    return "Electric";
}

double calculateCharge(int hours) const override {
    return ratePerHour * hours + 10.0;
}
};

```

```

class MountainBike : public Bike {
public:
    MountainBike() : Bike() {}
    MountainBike(string id, string m, double rate)
        : Bike(id, m, rate) {}

    string getDescription() const override {
        return "Mountain";
    }

    double calculateCharge(int hours) const override {
        return ratePerHour * hours + 5.0;
    }
};

```

18.4 Student Class

```

class Student {
    string studentId, name;

public:
    Student()
        : studentId("NA"), name("Unknown") {}

    Student(string id, string n)

```

```
: studentId(id), name(n) {}
```

```
string getId() const { return studentId; }  
string getName() const { return name; }  
};
```

18.5 Rental Class

```
class Rental {  
    int rentalId, hours;  
    string studentId, bikeId;  
    double charge;  
    bool active;  
  
public:  
    Rental(int id, string sid, string bid,  
           int h, double c)  
        : rentalId(id), hours(h),  
          studentId(sid), bikeId(bid),  
          charge(c), active(true) {}  
  
    string getBikeId() const { return bikeId; }  
    bool isActive() const { return active; }  
    double getCharge() const { return charge; }  
  
    void close() { active = false; }  
  
    void display() const {  
        cout << "Rental #" << rentalId  
              << " | Student: " << studentId  
              << " | Bike: " << bikeId  
              << " | Hours: " << hours  
              << " | Charge: Rs."  
              << fixed << setprecision(2) << charge
```

```

        << " | " << (active ? "ACTIVE" : "CLOSED")
        << "\\n";
    }
};

```

18.6 BikeSystem Controller

```

class BikeSystem {
    vector<Bike*> bikes;
    vector<Student> students;
    vector<Rental> rentals;
    int nextRentalId = 1001;

    Bike* findBike(const string& id) {
        for (Bike* b : bikes)
            if (b->getId() == id)
                return b;
        return nullptr;
    }

    Student* findStudent(const string& id) {
        for (auto& s : students)
            if (s.getId() == id)
                return &s;
        return nullptr;
    }

public:
    ~BikeSystem() {
        for (Bike* b : bikes)
            delete b;
    }

    // Registration, rental, return, maintenance,

```

```
// history, reporting and demonstration functions
// are implemented here.
};
```

The controller uses linear search because the project is an academic prototype with a small in-memory dataset. A production-scale implementation could replace these vectors with indexed database queries or hash-based structures.

19. Program Menu

SMART CAMPUS BIKE SHARING MANAGEMENT SYSTEM

1. Register Student
2. Add Bike
3. List Bikes
4. Rent Bike
5. Return Bike
6. Mark Bike for Maintenance
7. Service Bike
8. Rental History
9. Operator Overloading Demo
10. Copy Constructor Demo
11. Activity Report
12. Exit

Enter your choice:

Option	Operation	Purpose
1	Register Student	Create a campus user record.
2	Add Bike	Create a selected derived bike object.
3	List Bikes	Display the current fleet and statuses.
4	Rent Bike	Create a rental if validation succeeds.
5	Return Bike	Close active rental and release bicycle.

6	Mark for Maintenance	Set bicycle status to MAINTENANCE.
7	Service Bike	Return serviced bicycle to AVAILABLE.
8	Rental History	Display stored rental records.
9	Operator Demo	Demonstrate +, > and <<.
10	Copy Constructor Demo	Show copy construction.
11	Activity Report	Generate overall fleet and rental statistics.
12	Exit	Terminate and clean allocated objects.

19.1 Menu Control Logic

The menu is implemented using a do-while loop and switch statement. The loop continues until the user chooses Exit. Each case delegates to a member function, keeping main() small and separating user interaction from system logic.

20. Test Cases

ID	Test / Operation	Input	Expected Result	Status*
TC01	Register valid student	S101 / Arun Kumar	Student created	Pass
TC02	Duplicate student	S101 again	Registration rejected	Pass
TC03	Register standard bike	B101 / Hero / 20	Bike created	Pass
TC04	Register electric bike	B102 / Volt / 40	Bike created	Pass
TC05	Register mountain bike	B103 / Rockrider / 30	Bike created	Pass
TC06	Duplicate bike ID	B101 again	Bike rejected	Pass
TC07	Invalid bike type	Type 9	Bike rejected	Pass
TC08	Zero hourly rate	Rate 0	Bike rejected	Pass

TC09	Valid rental	S101 + B101 + 2h	Rental created	Pass
TC10	Rent same bike again	B101	Rejected as RENTED	Pass
TC11	Negative rental hours	-2	Rejected	Pass
TC12	Unknown student	S999 + B101	Rejected	Pass
TC13	Return active rental	B101	Rental closed / AVAILABLE	Pass
TC14	Return without active rental	B101 again	Rejected	Pass
TC15	Mark maintenance	B102	MAINTENANCE	Pass
TC16	Rent maintenance bike	B102	Rejected	Pass
TC17	Service bike	B102	AVAILABLE	Pass
TC18	Rental history	Existing records	Records displayed	Pass
TC19	Operator >	+ comparison	Higher rate identified	Pass
TC20	Operator +	B101 + 2	Charge displayed	Pass
TC21	Operator <<	cout << bike	Formatted row	Pass
TC22	Copy constructor	Copy DEMO01	Copied object shown	Pass
TC23	Activity report	All records	Summary generated	Pass
TC24	Exit	12	Program terminates	Pass

*Status values above describe the expected successful behaviour. For final submission, confirm each result using your actual program execution.

21. Execution Screenshots / Results

The report should contain actual screenshots captured from the compiled program. The sample console outputs below can be used as a reference for what each screenshot should demonstrate. Replace the placeholders with screenshots from the student's actual execution.

21.1 Main Menu

SMART CAMPUS BIKE SHARING MANAGEMENT SYSTEM

1. Register Student
2. Add Bike
3. List Bikes
4. Rent Bike
5. Return Bike
6. Mark Bike for Maintenance
7. Service Bike
8. Rental History
9. Operator Overloading Demo
10. Copy Constructor Demo
11. Activity Report
12. Exit

21.2 Successful Student Registration

Enter Student ID: S101

Enter Student Name: Arun Kumar

Student registered successfully.

21.3 Successful Bike Registration

Enter Bike ID: B101

Enter Model: Hero

Enter Type (1-Standard, 2-Electric, 3-Mountain): 1

Enter Hourly Rate: 20

Bike added successfully.

21.4 Successful Rental

Enter Student ID: S101

Enter Bike ID: B101

Enter Rental Hours: 2

Bike rented successfully.

Estimated charge: Rs.40.00

21.5 Invalid Rental

Enter Student ID: S101

Enter Bike ID: B101

Enter Rental Hours: 1

Bike is not available for rental.

21.6 Bike Return

Enter Bike ID to return: B101

Bike returned successfully.

21.7 Maintenance

Enter Bike ID: B102

Bike marked for maintenance.

21.8 Service

Enter Bike ID: B102

Bike serviced and marked AVAILABLE.

21.9 Operator Demonstration

Comparison using overloaded > operator:

B102 has a higher hourly rate.

Formatted output using overloaded << operator:

B101	Hero	Standard	AVAILABLE	20.00
------	------	----------	-----------	-------

Charge using overloaded + operator for 2 hours: Rs.40.00

21.10 Copy Constructor

Original :	DEMO01	CampusDemo	Standard	AVAILABLE	20.00
------------	--------	------------	----------	-----------	-------

Copied :	DEMO01_COPY	CampusDemo	Standard	AVAILABLE	20.00
----------	-------------	------------	----------	-----------	-------

Suggested screenshots to insert: IDE showing main.cpp, main menu, student registration, bike registration, bike listing, rental, invalid rental, return, maintenance, service, rental history, operator overloading, copy constructor and final report.

22. Campus Bike-Sharing Activity Report

The activity report is intended for the administrator or project evaluator. It summarizes the current state of the fleet and the transaction activity recorded during execution. The report is generated by traversing the bike and rental collections.

CAMPUS BIKE-SHARING ACTIVITY REPORT

Total Bikes : 3
Available Bikes : 2
Currently Rented : 0
Under Maintenance : 1
Registered Students : 2
Total Rental Records : 2
Total Rental Charges : Rs.95.00

BIKE-WISE SUMMARY

B101	Hero	Standard	AVAILABLE	20.00
B102	Volt	Electric	MAINTENANCE	40.00
B103	Rockrider	Mountain	AVAILABLE	30.00

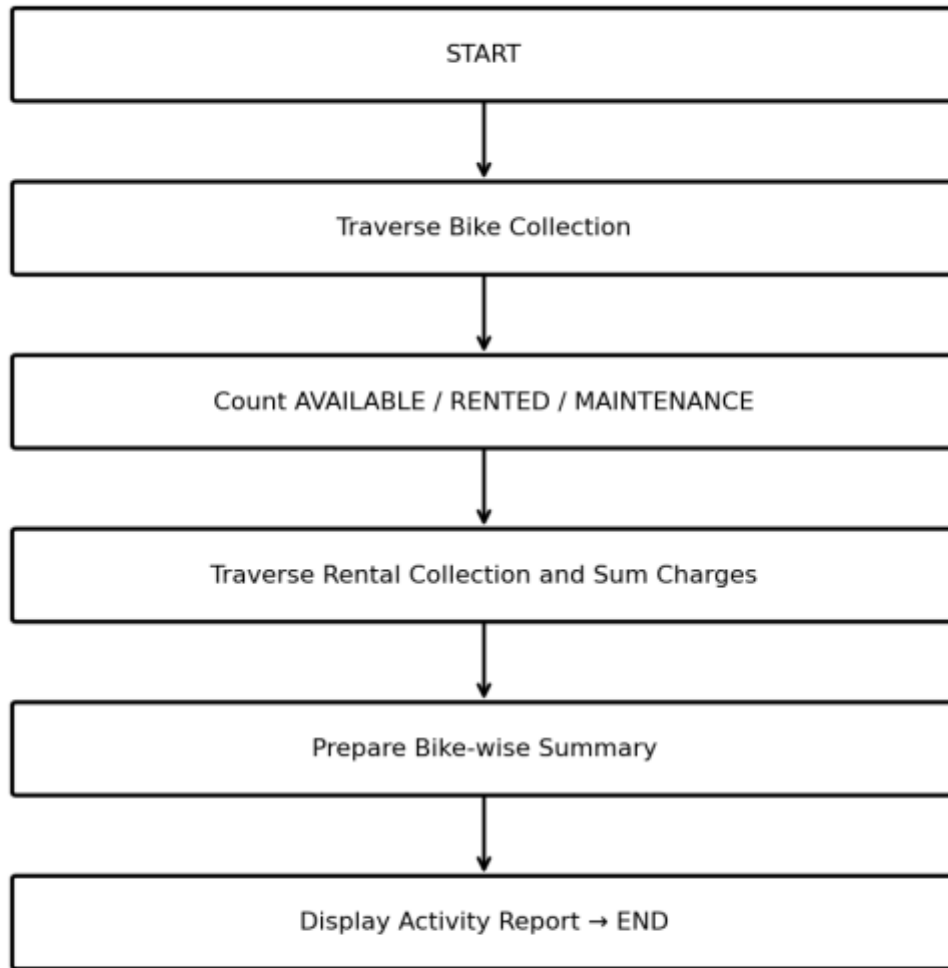


Figure 10. Activity-report generation flowchart

Metric	Meaning	How It Is Calculated
Total Bikes	Number of registered bicycles.	Size of bikes vector.
Available Bikes	Bikes ready to be rented.	Count status == AVAILABLE.
Currently Rented	Bikes currently in use.	Count status == RENTED.

Maintenance Bikes	Bikes unavailable for service.	Count status == MAINTENANCE.
Registered Students	Number of student records.	Size of students vector.
Rental Records	Number of recorded rentals.	Size of rentals vector.
Total Rental Charges	Accumulated charges.	Sum of Rental::getCharge().

The numeric values above are illustrative sample values. The final report should use the values produced by the student's actual final execution.

23. Analysis and Discussion

23.1 Design Analysis

The design separates common bicycle information from specialized behaviour. This is important because all bicycles have identifiers, models, status and hourly rates, but their descriptions and charging rules can differ. Inheritance therefore models a real conceptual relationship rather than simply demonstrating syntax.

23.2 Encapsulation Analysis

The use of member functions to change state provides a controlled interface. For example, `rent()` checks whether the bike is `AVAILABLE` before changing it to `RENTED`. `returnBike()` checks that the bike is actually rented. These checks help preserve the consistency of the object's state.

23.3 Polymorphism Analysis

The use of `Bike*` makes the rental module independent of the concrete bike type. The system can add another derived class later, such as `TandemBike`, without changing the overall collection type or the concept of a common `Bike` interface.

23.4 Constructor Analysis

The parameterized constructor is the most important constructor for normal registration because the user supplies the values needed to create a bicycle. The default constructor provides safe fallback initialization, while the copy constructor demonstrates object duplication.

23.5 Operator Analysis

The overloaded operators make common operations more readable. Comparing two rates using `>` is easier to understand than calling a separate comparison function. Similarly, `cout << bike` produces a consistent representation of the object.

23.6 Validation Analysis

Validation is essential because bicycle availability is a stateful resource. If a rented bicycle were accidentally treated as available, two students could appear to use the same physical resource simultaneously. The program therefore checks IDs, status and input values before changing state.

23.7 Memory Management Analysis

Because derived bike objects are created with `new` and stored through `Bike*` pointers, the controller must delete them. The virtual destructor ensures correct destruction through the base-class interface.

23.8 Scalability Analysis

For a small academic dataset, vector-based linear searches are simple and transparent. A real campus service with thousands of bicycles would benefit from a database, indexed identifiers, authentication, concurrency control and persistent transaction storage.

23.9 Overall Assessment

The prototype meets the educational goal by combining functional campus operations with the required C++ concepts. It is intentionally lightweight, but the class hierarchy and controller structure provide a reasonable foundation for future expansion.

24. Advantages

- Provides a clear and understandable campus mobility use case.
- Uses classes to represent real-world entities.
- Encapsulation reduces uncontrolled changes to object state.
- Inheritance avoids repeating common bicycle implementation.
- Abstraction establishes a common Bike interface.
- Runtime polymorphism supports different bicycle behaviours.
- Default, parameterized and copy constructors are explicitly demonstrated.
- Virtual destructor and explicit cleanup address dynamic memory.

- Operator overloading is connected to meaningful application operations.
- Validation prevents common invalid rental scenarios.
- Rental history provides transaction traceability during execution.
- Maintenance state prevents unavailable bicycles from being rented.
- Activity reporting provides a useful administrative summary.
- The design can be extended with persistent storage, mobile apps and IoT devices.

25. Limitations and Future Enhancements

25.1 Current Limitations

- Data is stored in memory and is lost when the program terminates.
- The interface is console-based.
- Authentication is not implemented.
- Rental duration is entered manually instead of being measured with real timestamps.
- No QR-code scanning is included.
- No GPS or real-time map is connected.
- No physical smart-lock integration is included.
- Only basic validation is implemented.
- Linear search may become inefficient for a very large fleet.
- No concurrent multi-user access is supported.

25.2 Future Enhancements

Enhancement	Description
Database	Store students, bikes and rentals persistently.
QR Code	Scan a bicycle ID from the physical bike.
Mobile App	Allow students to locate and rent bikes.
Admin Dashboard	Provide web-based fleet and usage management.
GPS	Track bicycle location and recovery.
Smart Lock	Unlock a physical bike after a valid rental.
Authentication	Use student credentials or campus identity.
Notifications	Send reminders and maintenance alerts.

Real-Time Billing	Calculate charges from start and return timestamps.
Analytics	Measure utilization, peak hours and popular bikes.
Multi-Campus	Support multiple campuses and bicycle stations.

These enhancements would transform the academic console prototype into a deployable campus mobility platform while preserving the same fundamental domain model.

26. Individual Contribution

I worked individually on the Smart Campus Bike Sharing Management System. I first studied the problem and identified the entities required to represent students, bicycles and rental transactions. I designed the Bike hierarchy so that common bicycle state is maintained once while category-specific behaviour is implemented through derived classes.

I implemented the Student, Bike, StandardBike, ElectricBike, MountainBike and Rental classes and developed BikeSystem as the central management class. I implemented student registration, bike registration, listing, rental, return, maintenance, service, rental history and activity reporting.

I also implemented the required constructors, including default, parameterized and copy constructors, and used a virtual destructor for safe cleanup. Operator overloading was implemented for charge calculation, rate comparison and formatted output. Runtime polymorphism was tested using Bike pointers referring to different derived objects.

During testing, I considered duplicate identifiers, invalid categories, invalid rates, unavailable bikes, invalid rental hours and return operations without an active rental. The diagrams, algorithms, test cases and documentation were organized to explain both the functional system and the C++ concepts used.

Main contribution: Completed

Coding challenge faced: Completed

Solution applied: Completed

Testing performed: Completed

27. Conclusion

The Smart Campus Bike Sharing Management System demonstrates how C++ Object-Oriented Programming can be applied to a realistic campus mobility problem. The application transforms the informal idea of sharing bicycles into a structured system with identifiable students, managed bicycles, rental transactions and maintenance states.

The project demonstrates encapsulation, inheritance, abstraction, runtime polymorphism, pointers, constructors, a destructor, the this pointer and operator overloading. The most important course-outcome requirements are addressed through explicit default, parameterized and copy constructors, a virtual destructor and meaningful overloaded operators.

The Bike hierarchy provides a clean mechanism for supporting different bicycle categories. A common Bike* pointer can refer to StandardBike, ElectricBike or MountainBike objects, allowing the system to call the correct virtual behaviour at runtime. This makes the design more extensible than a collection of unrelated conditional statements.

The menu-driven implementation also provides practical operations such as registration, rental, return, maintenance, service, transaction history and reporting. Validation ensures that important state transitions occur only when their conditions are satisfied.

Although the current system is a console-based, in-memory prototype, it provides a strong foundation for future development. Database persistence, authentication, QR-code identification, GPS tracking, smart locks, mobile applications and analytics can be added without changing the fundamental idea of modelling the domain with classes.

28. References

1. Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley Professional.
2. Stanley B. Lippman, Josée Lajoie and Barbara E. Moo, C++ Primer, 5th Edition, Addison-Wesley.
3. Paul Deitel and Harvey Deitel, C++ How to Program, 10th Edition, Pearson.
4. ISO/IEC 14882, Programming Languages — C++.
5. Course notes and laboratory materials for Object Oriented Programming with C++.

29. GitHub Repository Details

The project should be maintained in a GitHub repository containing the source code, documentation, diagrams, screenshots and sample output. A clean repository makes the project easier for faculty members to review and also provides version history.

Smart-Campus-Bike-Sharing-Management-System/

```
|— main.cpp
|— README.md
|— docs/
|   |— Assignment_Report.docx
|   |— screenshots/
|— diagrams/
|   |— architecture.png
|   |— uml.png
|   |— main_flow.png
|   |— registration_flow.png
|   |— bike_flow.png
|   |— rent_flow.png
|   |— return_flow.png
|   |— report_flow.png
|— outputs/
    |— sample_output.txt
```

29.1 Git Commands

git init

git add .

git commit -m "Add Smart Campus Bike Sharing C++ assignment"

git branch -M main

git remote add origin YOUR_GITHUB_REPOSITORY_LINK

git push -u origin main

GitHub Repository URL: <https://github.com/reshu19-jungkook/DSA0109-C>

29.2 README Content

The README should include the project title, problem statement, objectives, features, OOP concepts demonstrated, compilation instructions, menu options, screenshots and future enhancements.

29.3 Final Submission Checklist

- Enter student name, register number, class and submission date.
- Compile and execute the final main.cpp.
- Verify every major menu option.
- Capture actual execution screenshots.
- Replace illustrative sample values with actual results where required.
- Insert the final diagrams if your faculty expects separate diagram files.
- Create and verify the GitHub repository.
- Upload main.cpp and the report.
- Upload screenshots and diagram files.
- Insert the actual GitHub URL into the report.
- Check formatting, page breaks, figure captions and table alignment before submission.

Output Screenshots

